USBSID-Pico PCB firmware manual

Author: LouD - generated on 2026-10-24

Table of Contents

Disclaimer . . . 3
Firmware types . . . 3
Firmware naming . . . 3
  Board revision . . . 3
  Firmware naming . . . 3
  Firmware naming per Pico type . . . 4
  Naming examples for the rp2040 Pico1 . . . 4
  Naming examples for the rp2350 Pico2 . . . 4
  Firmwares with embedded USBSID-Player and Cynthcart . . . 4
How to flash your Pico . . . 5
Firmware configuration . . . 6
  Drivers . . . 6
  Linux Udev rules . . . 6
  Windows driver . . . 7
  USBSID-Configtool . . . 7
  Commandline configuration tool . . . 7
    Python Configuration GUI downloads . . . 7
    Linux / Windows downloads . . . 7
    Windows downloads . . . 7
  Configuration web page (for Chrome/Edge based browsers using webusb) . . . 8
  Default configuration (depends on the board version) . . . 8
  Board clock rate . . . 9
  Clock rate locked . . . 9
  Audio mode . . . 10
  Lock audio switch . . . 10
  SID Voltage & Configuration confirmation . . . 10
  Disable socket change detection . . . 11
  Socket settings . . . 11
    Socket One/Two Enabled . . . 11
    Socket One/Two Dualsid enabled . . . 11
    Socket One/Two Chiptype . . . 11
    Socket One/Two SID 1 type . . . 12
    Socket One/Two SID 2 type . . . 12
    Socket Two Act as One . . . 12

Flip Socket One and Socket Two . . . 12
Mix socket addresses (Quad SID only) . . . 12
Preset silent auto detection . . . 12
LED Enabled . . . 12
LED Idle breathe enabled . . . 12
RGBLED Enabled . . . 13
RGBLED Idle breathe enabled . . . 13
RGBLED Brightness . . . 13
RGBLED SID to use . . . 13
FMOpl Enabled . . . 13
FMOpl SID number . . . 13
CDC Enabled . . . 14
WebUSB Enabled . . . 14
Asid Enabled . . . 14
Midi Enabled . . . 14
WiFi Enabled . . . 14
WiFi state and IP address . . . 14
NSD Enabled . . . 15
Discovery Enabled . . . 15
Bluetooth NSD Enabled . . . 15
SID register reads . . . 15
SID register stream (FastReads) . . . 16
Debug logging . . . 16
License . . . 16
Software License ~ GNUv2 . . . 16
Configtool Software License ~ EPLv2 . . . 16
USBSID-Pico v1.5 PCB Hardware License ~ CC BY-NC-ND 4.0 . . . 16
USBSID-Pico v1.3 PCB Hardware License ~ CC BY-SA 4.0 . . . 17
USBSID-Pico v1.0 PCB Hardware License ~ CC BY-SA 4.0 . . . 17
USBSID-Volume-daughterboard v1.0 PCB Hardware License ~ CC BY-SA 4.0 . . . 17

Disclaimer

I do this stuff in my free time for my enjoyment. Since I like to share my joy in creating this with everyone I try my best to provide a working PCB and Firmware. I am in no way an electronics engineer and can give ***no guarantees*** that this stuff does not break or damage your hardware, computer, phone, or whatever you try to hook it up to. Be sure to take great care when inserting any real MOS SID chips into the board. While everything has been tested with real chips, this is in no way a guarantee that nothing could go wrong. Use of this board and firmware at your own risk! I am in no way responsible for your damaged hardware. That being said, have fun!

Firmware types







Each PCB revision is the firmware designator, please check your PCB revision, this is under the MOS logo and next to **USBSID-Pico** on your PCB.
Firmware filenames contain the PCB revision number e.g. `usbsidpico-rgb-v1.X.uf2` is for PCB revision `v1.0` and for use with a black clone Pico containing an RGB LED.

Firmware naming

Below are the filenames used for each Pico and board type, take great care in selecting the correct file for you Pico!
WARNING! Do *NOT* use the **RGB** firmware for any of the (non black) rp2040 or (non black) rp2350 Pico boards that do not contain an RGB LED, this can break your Pico!

Board revision

The 1.X in each filename equals for the PCB revision as mentioned under firmware types. Don't worry if you flash a firmware for the incorrect PCB revision, this causes no harm.

Firmware naming

`usbsidA-BC.uf2` examples: `usbsidpico-v1.0-rgb.uf2`, `usbsidpico2_w-v1.3.uf2` or `usbsidpico2-v1.5-rgb_em`.
A: The pico board type e.g. `pico`, `pico2`, `pico_2` or `pico2_w`

B: The PCB version e.g. v1.0, v1.3 or v1.5
C: A combination of optional features e.g. -rgb, _em or _net

Firmware naming per Pico type

Firmwares suitable for rp2040 Pico1 boards start with usbsidpico- Firmwares suitable for rp2040 Pico1_w boards start with usbsidpico_w-
Firmwares suitable for rp2350 Pico2 boards start with usbsidpico2-
Firmwares suitable for rp2350 Pico2_w boards start with usbsidpico2_w-

Naming examples for the rp2040 Pico1

Firmware naming suited for the pico 1 always start with usbsidpico-
usbsidpico-v1.X.uf2 for regular green rp2040 Pico boards.
usbsidpico-rgb-v1.X.uf2 for black clone rp2040 Pico boards with RGB LED onboard.
usbsidpico_w-v1.X.uf2 for regular green rp2040 PicoW boards.

NOTE! Due to the hardware limitations of the RP2040, highly demanding, cycle-exact SID tunes (dense per-cycle register writes, e.g. some digi/sample tunes) may not play at full speed on Pico1 or Pico1_w boards. Use a Pico2 or Pico2_w (rp2350) board for these tunes.

Naming examples for the rp2350 Pico2

Firmware naming suited for the pico 2 always start with usbsidpico2-
usbsidpico2-v1.X.uf2 for regular green rp2350 Pico2 boards.
usbsidpico2-rgb-v1.X.uf2 for black clone rp2350 Pico2 boards with RGB LED onboard.
usbsidpico2_w-v1.X.uf2 for regular green rp2350 Pico2W boards.

Firmwares with embedded USBSID-Player and Cynthcart

Firmwares ending with em have an embedded USBSID-Player that can play PSID and RSID tunes and PSID64 PRG/P00 files, and can also start Cynthcart via CC commands. Both now run through the same embedded player core.
For example usbsidpico-v1.0_em.uf2, usbsidpico2-v1.3-rgb_em.uf2 or usbsidpico2_w-v1.5_em_net.uf2
This USBSID-Player is still in active development and some tunes might not work or play at all. The embedded player needs a large chunk of RAM (over 80% on rp2040 boards, around 40% on rp2350 boards) and an overclocked CPU, but is now built for every Pico type, rp2040 included. On pico2_w it is only ever offered bundled with Network SID Device support (_em_net), since that board has RAM to spare for both together.
WARNING! Whenever the embedded player or Cynthcart is used the Pico MCU is overclocked to 250MHz, regardless of Pico type. Passive cooling _might/could be needed.
See the USBSID-Player-manual and USBSID-Pico-Cynthcart-manual files for more information.

How to flash your Pico

NOTE:
When flashing a new firmware version, all previously configured settings will be reset to default. Use the commandline configtool to save your current settings to a `ini` file if you want to save them!
A Raspberry Pi Pico board is incredibly easy to flash, as it comes with a built in bootloader for flashing new firmwares in the `uf2` format.
In order to flash a new firmware to your USBSID-Pico you will need to put the Pico into bootloader mode.
This can be done in multiple ways:

With the cable already plugged into your computer and Pico and seated on the board (using buttons):

- Press and hold the `BOOTSEL` button on the Pico.
- Press and release the `RST` button on the USBSID-Pico board.
- Now release the `BOOTSEL` button.
- A new drive should appear on your computer called `RPI-RP2` or `RP2350` depending on your Pico type.
- Copy the correct `uf2` firmware file to this directory.
- After copying the Pico will reboot and your Pico is flashed.

With the cable not plugged in to your computer and Pico and also not seated on the board (using buttons):

- Plug in the USB cable to your Pico and not into your computer.
- While holding the `BOOTSEL` button on the Pico plugin the other end of the USB cable into your computer.
- Now release the `BOOTSEL` button.
- A new drive should appear on your computer called `RPI-RP2` or `RP2350` depending on your Pico type.
- Copy the correct `uf2` firmware file to this directory.
- After copying the Pico will reboot and your Pico is flashed.

With the cable already plugged into your computer and Pico and seated on the board (using software):

- With the GUI config tool
- Go to the `DEBUG` tab and click on `BOOTLOADER`
- A new drive should appear on your computer called `RPI-RP2` or `RP2350` depending on your Pico type.
- Copy the correct `uf2` firmware file to this directory.
- After copying the Pico will reboot and your Pico is flashed.

or

- Go to the config webpage (usbsid.loudai.nl/index.html?player=webusb&debug=usbsidpico):
- Connect to the board using `WebUSB (Hermit)` as emulator
- Click on `CONFIG` to open the configuration tab
- Scroll to the bottom of the page and click on `BOOTLOADER` in the box named `DEBUG`
- A new drive should appear on your computer called `RPI-RP2` or `RP2350` depending on your Pico type.
- Copy the correct `uf2` firmware file to this directory.
- After copying the Pico will reboot and your Pico is flashed.

or

- With the CLI config tool command in your `PATH` variable
- Type `cfg_usbsid -boot` to send the reboot to bootloader command
- A new drive should appear on your computer called `RPI-RP2` or `RP2350` depending on your Pico type.
- Copy the correct `uf2` firmware file to this directory.
- After copying the Pico will reboot and your Pico is flashed.

Firmware configuration

USBSID-Pico has an extensive set of configuration options that expands with each firmware release.

Drivers

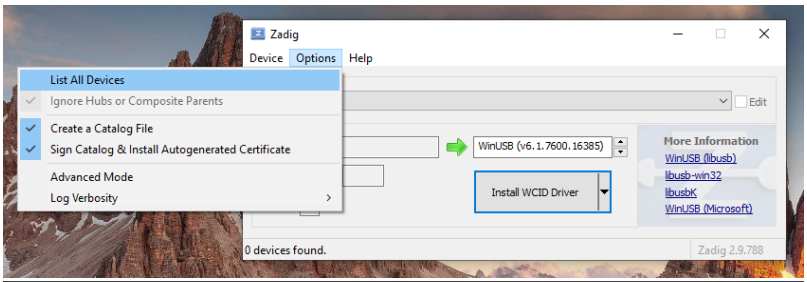
Linux Udev rules

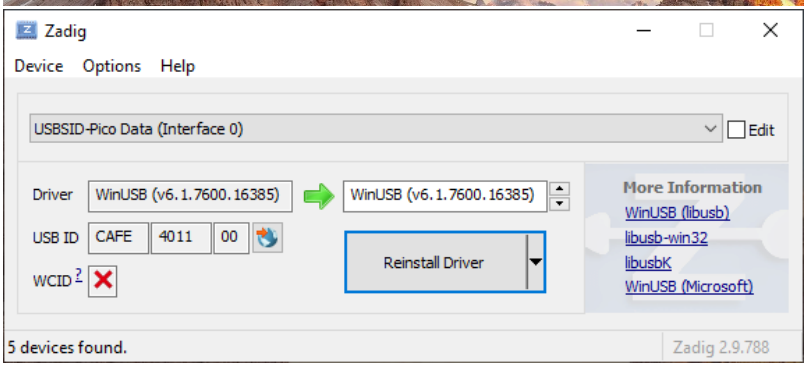
In the examples/udev directory you can find the udev rules that I use on Linux.
This purely an example file that you can use and change to your own needs. Steps required for this to work

```
# Check if you are in the plugdev group with the groups command
groups
# If not, add yourself to the plugdev group, then log out and in
sudo usermod -aG plugdev $USER
# Copy the udev ules file to the correct directory
sudo cp 69-usbsid-permissions.rules /etc/udev/rules.d
# Now reload the udev rules
udevadm control --reload-rules && udevadm trigger
# Not working? Try reloading the service
sudo systemctl restart udev
```

Windows driver

Use Zadig to install the correct driver. Replace the driver for `USBSID-Pico Data` with WinUSB.





USBSID-Configtool

You can configurate your board by using the official USBSID-Configtool
The tool is available here USBSID-Configtool in the releases section

Commandline configuration tool

You can configurate your board with: - the commandline config-tool - the commandline config-tool with the Python Configuration GUI by ISL/Samar if you prefer a GUI.

Python Configuration GUI downloads

Download script
Python3 → Download
TkInter → Installation guide
And either one of the Linux or Windows CLI downloads below

Linux / Windows downloads

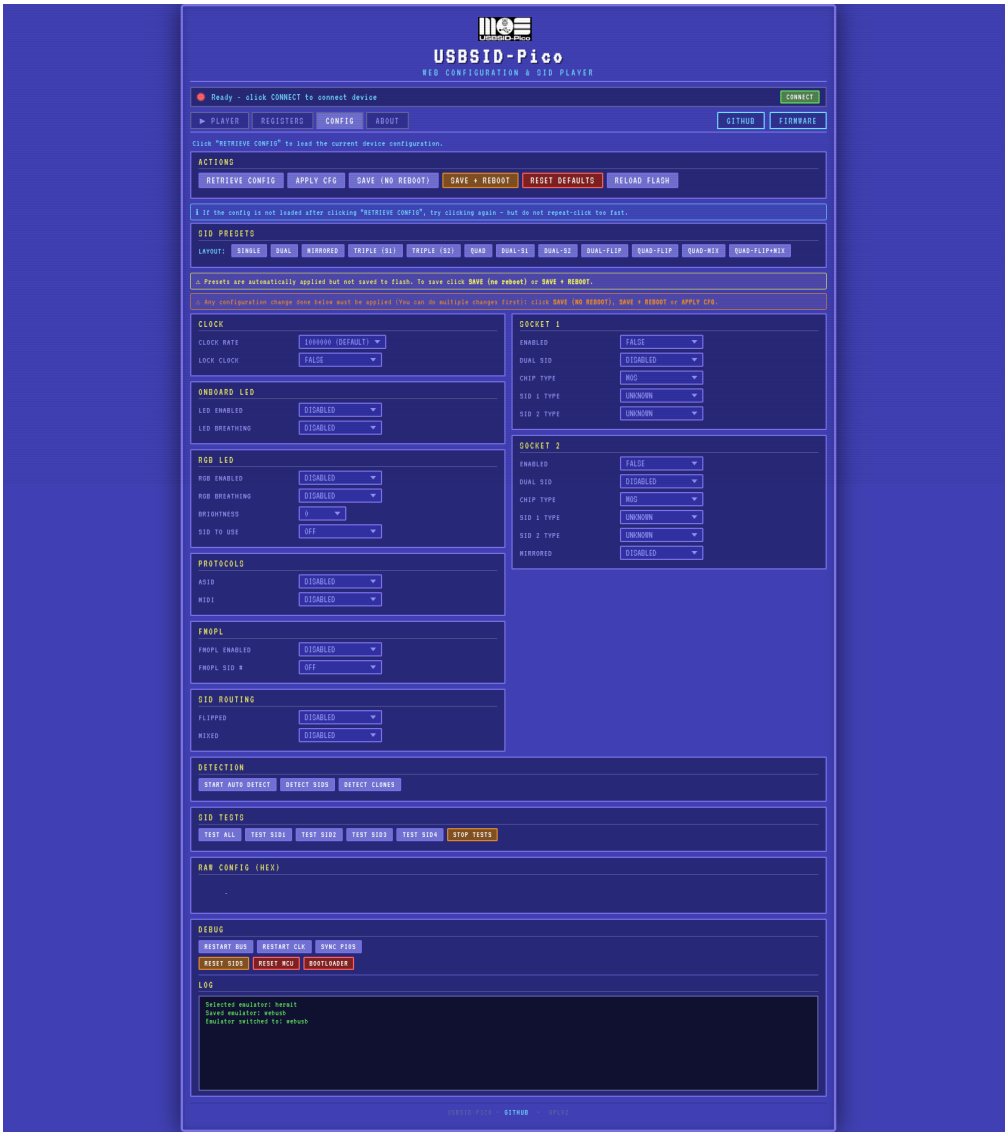
The latest version of the CLI config tool is available in a zip file called `tools.zip` from release v0.6.0-BETA and up

Windows downloads

If the DLL's are missing from the zip file you can always download them from here:
Download DLL 1
Download DLL 2

Configuration web page (for Chrome/Edge based browsers using webusb)

USBSID web configuration tool (requires a Chrome based browser).
If needed you can change your USBSID configuration after selecting WebUSB and clicking on `Open config`.



The player is set up with borrowed code from Deepsid using Hermit's JsSID implementation.

Default configuration (depends on the board version)

The default configuration differs slightly between Pico1 (rp2040) and Pico2 (rp2350) boards.
An explanation for these settings can be found further on in this document.
Board clock rate: `1000000Hz (DEFAULT)` for rp2040, `985248Hz (PAL)` for rp2350
Clock rate locked: `False`
Audio mode: `Mono`
Preset silent auto detection: `False` (`True` for PCB revision v1.5+)
Socket One Enabled: `True`
Socket One Dualsid enabled: `False`
Socket One Chiptype: `Unknown`
Socket One SID 1 type: `Unknown`

Socket One SID 2 type: N/A
Socket Two Enabled: True
Socket Two Dualsid enabled: False
Socket Two Chiptype: Unknown
Socket Two SID 1 type: Unknown
Socket Two SID 2 type: N/A
Socket Two Act as One: False
Flip Socket One and Socket Two: False
Mix socket addresses (Quad SID): False
LED Enabled: True
LED Idle breathe enabled: True
RGBLED Enabled: False (True for RGB Pico's)
RGBLED Idle breathe enabled: False (True for RGB Pico's)
RGBLED Brightness: 0 (127 for RGB Pico's)
RGBLED SID to use: -1 (1 for RGB Pico's)
CDC Enabled: True
WebUSB Enabled: True
Asid Enabled: True
Midi Enabled: True
WiFi Enabled: False (_w boards only)
NSD Enabled: True (_w boards only, needs WiFi Enabled)
Discovery Enabled: True (_w boards only)
Bluetooth NSD Enabled: False (_w boards only)
NSD port: 6581 (_w boards only)
Hostname: USBSID-Pico (_w boards only)
FMOpl Enabled: False
FMOpl SID no: 0

Board clock rate

The Commodore64 has different clockrates depending on where you live, USBSID-Pico supports these different clock rates as configuration setting.
Supported SID players will automatically send a new clock rate request to the board to switch to.
Available options are:
DEFAULT 1000000Hz, PAL 985248Hz, NTSC 1022727, DREAN 1023440, NTSC2 1022730

Clock rate locked

Some players start sending data directly after the clockrate is changed. While in most cases this is no issue, some SID clones need some time to start up. This causes issues when the clock rate is changed on the fly.
Also changing the clock rate multiple times after another can cause data corruption to the SID which causes garbled sound.
To overcome this it is possible to lock the clockrate to the current setting it is set to. This blocks the ability for on the fly change requests.
Available options are:
False, True

Audio mode

PCB revision v1.3 only
Sets the audio mode for the headphone jack and audio out pins.
When set to `Mono` the audio of both SID sockets will be combined to mono.
When set to `Stereo` the audio of both SID sockets will be separated into left for `SID1` and right for `SID2`
Available options are:
`Mono`, `Stereo`

Lock audio switch

PCB revision v1.3 only
Locks the audio switch into its current state (`Mono` or `Stereo`) and prevents it from being changed.
Available options are:
`False`, `True`

SID Voltage & Configuration confirmation

PCB revision v1.5 only
The board boots with 5v and 9v enabled to automatically detect any changes in the socket / chip configuration, it does this by running a chip and SID detection.
When a change is detected the boards turns off the 9v and 5v and the onboard (RGB)LED starts flashing, making the board unusable until the configuration is validated and confirmed.
Please verify the configuration with care before confirming, failure to do so might damage/kill your SID chips, and we don't want that!
There are multiple ways to validate and confirm the configuration:
GUI Configtool
On opening the configtool and connecting to the board there will be a text stating the configuration needs to be confirmed. Verify the config on the `SOCKETS` tab and when done click `CONFIRM`



Web Configtool
On opening the config website and connecting to the board there will be a text stating the configuration needs to be confirmed. Verify the config on the `CONFIG` tab by clicking `RETRIEVE CONFIG` first and when done validating click `CONFIRM CONFIG`. Both buttons will glow when a click is needed



CLI Configtool
First read the configuration with `cfg_usbsid -r`, the printed configuration will have a line that states the config needs confirmation. You can do this with `cfg_usbsid -ack`.

Disable socket change detection

PCB revision v1.5 only
When you have a fixed socket setup, e.g. you do not plan on changing the chips anytime soon, it is possible to disable the change detection on boot. This will make the board start up with 5v and 9v and then - if 12v is required - switch to the required voltage setting during the boot process by switching the voltage back off, setting the correct hv voltage and then switching it back on.
When the socket detection *is* on, the board will *always* try to detect any changes in the configuration *before* the required voltage is applied.
By setting the `Disable socket detection on boot` option you can control the above behaviour:
GUI Configtool



Web Configtool



CLI Configtool
Change the behaviour with `cfg_usbsid -sad N` where N can be 0 (disabled) or 1 (enabled)

Socket settings

Each socket comes with its own set of configurable options.

Socket One/Two Enabled

Enable or disable the use of this socket.
Available options are:
`False`, `True`

Socket One/Two Dualsid enabled

Enable or disable dualsid for this socket.
This requires the `Chiptype` for this socket to be set to any clone chip (not `MOS`) and ofcourse a dual SID supporting SID clone with support of connecting the A5 address line.
Available options are:
`False`, `True`

Socket One/Two Chiptype

Set the type of SID chip for this socket.
Available options are:
`MOS` (real SID), `Unknown`, `SKPico`, `ARMSID`, `ARM2SID`, `FPGASID`, `RedipSID`, `PDSID`, `BackSID`, `SIDEmu`

Socket One/Two SID 1 type

Available options are:
`Unknown`, `N/A`, `8580`, `6581`, `FMOpl`

Socket One/Two SID 2 type

Available options are:
`Unknown`, `N/A`, `8580`, `6581`, `FMOpl`

Socket Two Act as One

When enabled, this will disable other Socket Two settings and this socket will receive all writes from Socket One as it were mirrored.
Available options are:
`False`, `True`

Flip Socket One and Socket Two

When enabled, Socket One and Socket Two are swapped so Socket One's physical position responds as Socket Two and vice versa.
Available options are:
`False`, `True`

Mix socket addresses (Quad SID only)

When enabled, addresses for a 4-SID setup are mixed between sockets instead of being assigned sequentially per socket. Combine with `Flip` for the remaining address orderings.
Available options are:
`False`, `True`

Preset silent auto detection

When enabled, the board silently re-runs chip/SID auto detection before applying a preset.
Default is `False` on PCB revision v1.0~v1.3 and `True` on PCB revision v1.5+.
Available options are:
`False`, `True`

LED Enabled

Disable or enable the use of the onboard LED.
When enabled this automatically enables the LED as Vu meter for the voices of `SID1`.
Available options are:
`False`, `True`

LED Idle breathe enabled

Disable or enable the idle breathing effect for the onboard LED.

Available options are:
`False`, `True`

RGBLED Enabled

Only for black Pico clones with onboard RGB LED
Disable or enable the use of the onboard RGBLED.
When enabled this automatically enables the RGBLED as Vu meter.
Available options are:
`False`, `True`

RGBLED Idle breathe enabled

Only for black Pico clones with onboard RGB LED
Disable or enable the idle breathing effect for the onboard RGBLED.
Available options are:
`False`, `True`

RGBLED Brightness

Only for black Pico clones with onboard RGB LED
Set the maximum brightness of the onboard RGBLED.
Available range is:
`0 to 255`

RGBLED SID to use

Only for black Pico clones with onboard RGB LED
Set the SID of which to use the voices for the Vu meter.
Available options are (depending on your SID configuration):
`-1` off, `0` off, `1` SID1, `2` SID2, `3` SID3, `4` SID4

FMOpl Enabled

Requires a SID clone with FMOpl capabilities, the socket `Chip type` set to `Clone` and `SID type` set to `FMOpl` before activation!
Available options are:
`False`, `True`

FMOpl SID number

This setting cannot be changed manually This setting will automatically be set to the correct SID number when all requirements are met as described under `FMOpl Enabled`

CDC Enabled

This config setting is intended for future use Available options are:
`False`, `True`

WebUSB Enabled

This config setting is intended for future use Available options are:
`False`, `True`

Asid Enabled

Disable or enable ASID protocol handling (SysEx-based SID playback over MIDI).
Available options are:
`False`, `True`

Midi Enabled

Disable or enable MIDI input handling (note/CC/voice control of the SID(s)).
Available options are:
`False`, `True`

WiFi Enabled

`pico_w`/`pico2_w` boards firmware with _net only
No GUI/Web configtool support yet, use the `cfg_usbsid` commandline configtool: `-ssid`, `-psk`, `-host`, `-wen`, `-wapply`
Enables the board's WiFi station connection. Requires an SSID and password to have been provisioned first.
Available options are:
*`False*, `True`

WiFi state and IP address

`pico_w`/`pico2_w` boards firmware with _net only
Two read-only queries report the WiFi connection:
`WIFI_STATUS` replies with 2 bytes: the link (`0` down, `1` connected) and the NSD session (`0` no session, `1` active). A byte is `0xFF` when the firmware is not built with WiFi (link) or NSD (session).
`WIFI_READ_IP` replies with 5 bytes: the WiFi state followed by the IPv4 address, first octet first. The state is `0` off (WiFi not started), `1` down (no link, waiting to connect), `2` connecting, `3` connected, or `0xFF` when the firmware is not built with WiFi. The address bytes are all zero unless the state is `3`.
The commandline configtool prints both with `cfg_usbsid -wifis`, and `cfg_usbsid -rnc` prints the IP address with the network config.

NSD Enabled

`pico_w`/`pico2_w` boards firmware with _net only
No GUI/Web configtool support yet, use the `cfg_usbsid` commandline configtool: `-nsden`, `-nsdp`, `-wapply`
Enables the Network SID Device (NSD) server: a TCP listener (default port `6581`) plus a UDP discovery responder, letting NSD clients such as ACID 64, JSidplay2 and SIDPlay find and drive the board over the LAN exactly like a native NSD server. Only one NSD session (WiFi or Bluetooth) can be active at a time; USB always keeps priority over the SID bus.
Changes to `NSD Enabled` and `Discovery Enabled` apply at once, a reboot is not needed. Disabling NSD closes the listener and ends a running TCP NSD session. A new port applies on `-wapply`, which also saves the network config. When the listener cannot bind its port it retries every second.
The NSD server buffers writes in a ring of 4096 entries and reports `BUSY` to the client above about 160 ms of queued cycles (PAL), it accepts writes again below about 120 ms. Read-only config queries (`READ_CONFIG`, `READ_NETCFG`, `WIFI_STATUS`, `WIFI_READ_IP`, version and feature queries and similar) do not take the SID bus, a host polling the board does not hold off a running NSD session.
Available options are:
`False`, `True`

Discovery Enabled

`pico_w`/`pico2_w` boards firmware with _net only
No GUI/Web configtool support yet, use the `cfg_usbsid` commandline configtool: `-discen`, `-wapply`
Enables the UDP discovery responder that answers NSD clients searching the LAN for a board. It always listens on UDP port `6581`, whatever TCP port `NSD Enabled` uses. Applies at once.
Available options are:
`False`, `True`

Bluetooth NSD Enabled

`pico_w`/`pico2_w` boards firmware with _net only
No GUI/Web configtool support yet, use the `cfg_usbsid` commandline configtool: `-bten`
Enables the Bluetooth Classic SPP transport for the same Network SID Device protocol as `NSD Enabled` above. No NSD client speaks Bluetooth SPP directly, so this needs a small host-side RFCOMM-to-TCP bridge (see `examples/nsd_spp_bridge.py`) to be usable with existing players.
Available options are:
`False`, `True`

SID register reads

Per the SID datasheet only registers `$19` to `$1C` are readable. A real or unknown chip answers a read of any other register with the value left on the data bus. A read returns that value to the host, but the board's register mirror (used by the onboard emulator and the SID register stream) keeps the last written value of that register. Clone chips keep full register readback. In mirrored mode (`Socket Two Act as One`) both sockets must hold a clone chip for this.

SID register stream (FastReads)

Optional build feature, off in the default builds
Firmware built with `-DENABLE_SID_STREAM=1` adds a second vendor USB interface (interface `7`, bulk endpoints `0x07` and `0x87`) that streams SID register frames to the host: one 63 byte frame per enabled SID per tick, up to 1000 ticks per second. The stream is started and stopped with vendor control requests and stops on unmount and suspend. The build reports the feature in bit 6 of the `US_FEATURES` byte and sets `bcdDevice` to `0x0101`, default builds have no interface 7 and `bcdDevice 0x0100`.
The frames are copied from the board's register mirror, the stream does not read from the SID chips.
Batch reads are part of every build and do not need this option: one `READ` packet with a register count reads up to 63 registers in one USB round trip. Hosts detect it by firmware version 0.8.1 or newer.

Debug logging

The CDC and vendor (WebUSB) event logs (mount, line state, line coding, break, vendor TX and control requests) print only when the build has USB IO debugging enabled (`USBIO_DEBUGGING` in `CMakeLists.txt`, default `0`). Earlier builds printed them over UART regardless, each log line stalled playback for milliseconds.

License

Software License ~ GNUv2

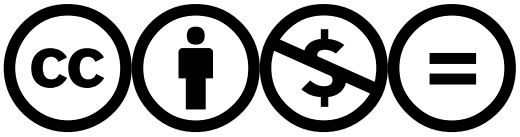
All code written by me in this repository is licensed under the terms of the GNU General Public License as published by the Free Software Foundation, version 2. Any code in this repository that is not written by me automatically falls under the licensing conditions by the authors of said code as mentioned in the source code header. For for more information about this license please read the LICENSE document in the root of the USBSID-Pico repository.

Configtool Software License ~ EPLv2

This program and the accompanying materials are made available under the terms of the Eclipse Public License 2.0 which is available at eclipse.org.
SPDX-License-Identifier: EPL-2.0

USBSID-Pico v1.5 PCB Hardware License ~ CC BY-NC-ND 4.0

USBSID-Pico v1.5 PCB © 2026 by LouD is licensed under
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International



USBSID-Pico v1.3 PCB Hardware License ~ CC BY-SA 4.0

USBSID-Pico v1.3 PCB © 2025-2026 by LouD is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



USBSID-Pico v1.0 PCB Hardware License ~ CC BY-SA 4.0

USBSID-Pico v1.0 PCB © 2024-2026 by LouD is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



USBSID-Volume-daughterboard v1.0 PCB Hardware License ~ CC BY-SA 4.0

Volume-daughterboard v1.0 PCB © 2026 by LouD is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



Author: LouD - generated on 2026-10-24